OAVS TGT COMPUTER TEACHER

USING DEBUGGING TOOLS

Visual Basic 6.0 | MCQ Question Bank

128 exam-oriented questions with answers and explanations

Syllabus Covered

a. Types of errors & debug menu: types of errors; debug menu
b. Testing the application: Immediate window; debug and Locals window; setting watch expressions
c. Implementing error handler: how VB handles runtime errors; VB error handler
d. VB error-handling options: how VB handles runtime errors; disabling the error handler
e. Inline error handling

No.	Section	Questions
1	Types of Errors	Q1-Q10
2	Debug Menu	Q11-Q25
3	Immediate Window	Q26-Q36
4	Debug and Locals Windows	Q37-Q47
5	Watch Expressions	Q48-Q57
6	Testing and Debugging Applications	Q58-Q67
7	Runtime Errors and Error Handlers	Q68-Q87
8	VB Error-Handling Options	Q88-Q93
9	Disabling Error Handlers	Q94-Q98
10	Inline Error Handling	Q99-Q106
11	Important and Tricky Questions	Q107-Q118
12	Mixed OAVS TGT Practice Set	Q119-Q128

Section 1 10 questions

Types of Errors

Q1 | Concept | Debugging Tools

Which type of error occurs when a statement violates the grammar rules of Visual Basic, for example a misspelled keyword or a missing `Then` in an `If` statement?

A. Overflow error
B. Runtime error
C. Logical error
D. Syntax error

Ans: D

Explanation: A syntax error breaks the rules of the language and is reported by the editor or compiler before the program can run.

Q2 | Concept | Debugging Tools

Which type of error is generally the hardest to detect, because VB shows no error message and the program keeps running?

A. Runtime error
B. Syntax error
C. Compile error
D. Logical error

Ans: D

Explanation: A logical error gives wrong results without any warning. It is found only by testing the output and tracing the code.

Q3 | Error ID | Debugging Tools

The code below is syntactically correct and compiles without any warning. What kind of error occurs when it is executed?

```
Dim a As Integer, b As Integer
a = 20
b = 0
MsgBox a / b
```

A. Logical error with a wrong result
B. Compile error (Variable not defined)
C. Runtime error (Division by zero)
D. Syntax error

Ans: C

Explanation: Division by zero is detected only when the statement executes, so VB raises run-time error 11.

Q4 | Concept | Debugging Tools

Which of the following is reported as a compile error when `Option Explicit` is in effect?

A. Using a variable that has not been declared
B. Assigning the text 'abc' to an Integer variable at run time
C. Writing a wrong formula such as `Total = Qty + Rate`
D. Dividing a number by zero

Ans: A

Explanation: Option Explicit forces every variable to be declared. An undeclared name gives the compile error 'Variable not defined'.

Q5 | Concept | Debugging Tools

Which setting on the Editor tab of Tools > Options makes VB check the syntax of each line as soon as Enter is pressed?

A. Require Variable Declaration
B. Auto Syntax Check
C. Auto List Members
D. Auto Data Tips

Ans: B

Explanation: With Auto Syntax Check on, VB displays a message and flags the line as soon as a syntax error is entered.

Q6 | Error ID | Debugging Tools

A text value such as 'Hello' is assigned to an Integer variable at run time. Which run-time error is raised?

A. Subscript out of range (Error 9)
B. Division by zero (Error 11)
C. Type mismatch (Error 13)
D. Overflow (Error 6)

Ans: C

Explanation: Type mismatch (Error 13) occurs when a value is not compatible with the data type it is assigned to.

Q7 | Code Output | Debugging Tools

What happens when the following code is executed?

```
Dim a As Integer, b As Integer, c As Integer
a = 30000
b = 5000
c = a + b
```

A. c becomes 35000
B. Compile error: Variable not defined
C. Run-time error 6 (Overflow)
D. Run-time error 13 (Type mismatch)

Ans: C

Explanation: Integer + Integer is evaluated as Integer. 35000 exceeds the Integer maximum of 32767, so Error 6 (Overflow) is raised.

Q8 | Statement-Based | Debugging Tools

Which statement about syntax errors is TRUE?

A. They must be corrected before the program can run
B. They occur only while the program is running
C. They are detected only by supplying test data
D. They never produce any message

Ans: A

Explanation: VB will not run code that contains syntax errors; they are flagged in the editor or when the code is compiled.

Q9 | Concept | Debugging Tools

Which type of error may remain hidden until the faulty statement is actually executed, for example in a rarely used procedure?

A. Runtime error
B. Misspelled keyword typed in the code window
C. Missing parenthesis in an expression
D. Syntax error flagged by Auto Syntax Check

Ans: A

Explanation: Run-time errors depend on the data and the execution path, so they appear only when the faulty statement is reached.

Q10 | Concept | Debugging Tools

What is the main advantage of using Run > Start With Full Compile (Ctrl+F5)?

A. It converts the project into an .exe file
B. It removes all breakpoints from the project
C. It makes the program run faster by skipping error checks
D. It finds compile errors in all procedures before the program starts

Ans: D

Explanation: A normal run compiles on demand, so some compile errors show up only when a procedure is reached. Full compile checks all code first.

Section 2 15 questions

Debug Menu

Q11 | Concept | Debugging Tools

Which VB 6.0 menu contains the commands Step Into, Step Over, Step Out and Run To Cursor?

A. Tools
B. Debug
C. Run
D. Edit

Ans: B

Explanation: The Debug menu groups the stepping, breakpoint and watch commands used to trace code in break mode.

Q12 | Concept | Debugging Tools

Which key executes the next statement and also steps into any procedure that the statement calls?

A. F8 (Step Into)
B. Ctrl+F8 (Run To Cursor)
C. F9 (Toggle Breakpoint)
D. Shift+F8 (Step Over)

Ans: A

Explanation: Step Into (F8) executes one statement at a time and traces inside called procedures.

Q13 | Concept | Debugging Tools

Step Over (Shift+F8) differs from Step Into because it:

A. Executes a called procedure as a single step without tracing through it
B. Skips the current statement without executing it
C. Returns control to the calling procedure
D. Runs the program until the next breakpoint

Ans: A

Explanation: Step Over runs the whole called procedure at once and stops at the next statement of the current procedure.

Q14 | Concept | Debugging Tools

Which Debug command runs the rest of the current procedure and stops in the calling procedure?

A. Run To Cursor
B. Set Next Statement
C. Step Over
D. Step Out

Ans: D

Explanation: Step Out (Ctrl+Shift+F8) finishes the current procedure and returns to the statement after the call in its caller.

Q15 | Concept | Debugging Tools

Which Debug command executes code from the current point up to the line where the text cursor is placed and then enters break mode?

A. Show Next Statement
B. Run To Cursor
C. Quick Watch
D. Toggle Breakpoint

Ans: B

Explanation: Run To Cursor (Ctrl+F8) acts like a temporary breakpoint at the cursor line, saving repeated stepping.

Q16 | Concept | Debugging Tools

Pressing F9 on an executable line in the Code window will:

A. Set or remove a breakpoint on that line
B. Run the application
C. Display the Quick Watch dialog
D. Step into the next procedure

Ans: A

Explanation: F9 is the shortcut for Toggle Breakpoint. Execution pauses before the marked line is executed.

Q17 | Concept | Debugging Tools

Which Debug command lets you skip or repeat statements by moving the execution point to another line of the same procedure?

A. Set Next Statement
B. Clear All Breakpoints
C. Add Watch
D. Step Out

Ans: A

Explanation: Set Next Statement (Ctrl+F9) moves the yellow arrow to the chosen line, which will be executed next. It works only within the current procedure.

Q18 | Concept | Debugging Tools

In break mode, which command brings back the highlighted line that will execute next after you have scrolled elsewhere in the code?

A. Run To Cursor
B. Show Next Statement
C. Quick Watch
D. Edit Watch

Ans: B

Explanation: Show Next Statement displays the statement marked with the yellow arrow that will run next.

Q19 | Concept | Debugging Tools

Which keyboard shortcut opens the Quick Watch dialog box in break mode?

A. Ctrl+Shift+F9
B. Shift+F8
C. Shift+F9
D. Ctrl+F9

Ans: C

Explanation: Quick Watch is Shift+F9. Ctrl+F9 is Set Next Statement and Ctrl+Shift+F9 is Clear All Breakpoints.

Q20 | Concept | Debugging Tools

Which key combination removes all the breakpoints that have been set in the project?

A. Ctrl+F9
B. Ctrl+F8
C. Ctrl+Shift+F9
D. Shift+F9

Ans: C

Explanation: Clear All Breakpoints (Ctrl+Shift+F9) removes every breakpoint at once; F9 toggles only the current line.

Q21 | Concept | Debugging Tools

On which of the following lines can a breakpoint NOT be set?

A. A comment line or a variable declaration line
B. A line that calls another procedure
C. A line containing a MsgBox call
D. An assignment statement

Ans: A

Explanation: Breakpoints can be set only on executable statements, not on comments or declarations.

Q22 | Concept | Debugging Tools

What happens to the breakpoints when a VB 6.0 project is closed?

A. They are saved in the form's .frm file
B. They are stored permanently in the Windows registry
C. They are saved in the .vbp project file
D. They are not saved and are lost

Ans: D

Explanation: Breakpoints are temporary debugging aids that last only for the current IDE session.

Q23 | Concept | Debugging Tools

Which of the following is NOT a command of the Debug menu?

A. Add Watch
B. Toggle Breakpoint
C. Clear All Breakpoints
D. Start With Full Compile

Ans: D

Explanation: Start With Full Compile belongs to the Run menu. Add Watch, Toggle Breakpoint and Clear All Breakpoints are Debug menu commands.

Q24 | Concept | Debugging Tools

Which keyboard shortcut opens the Edit Watch dialog box for an existing watch expression?

A. Ctrl+G
B. Shift+F9
C. Ctrl+W
D. Ctrl+L

Ans: C

Explanation: Debug > Edit Watch (Ctrl+W) modifies the expression, context or watch type of a watch. Shift+F9 is Quick Watch, Ctrl+L is Call Stack and Ctrl+G is Immediate Window.

Q25 | Concept | Debugging Tools

Which Debug menu command is used to define a new watch expression, choosing its context and watch type?

A. Toggle Breakpoint
B. Add Watch
C. Step Over
D. Quick Watch

Ans: B

Explanation: Add Watch opens a dialog for the expression, context and watch type. Quick Watch only shows a one-time value.

Section 3 11 questions

Immediate Window

Q26 | Concept | Debugging Tools

Which window in Visual Basic 6.0 allows a programmer to evaluate expressions and execute statements during debugging?

A. Properties Window
B. Immediate Window
C. Project Explorer
D. Form Layout Window

Ans: B

Explanation: The Immediate Window allows expressions to be evaluated and statements to be executed while debugging an application.

Q27 | Concept | Debugging Tools

Which keyboard shortcut opens the Immediate Window in the VB 6.0 IDE?

A. Ctrl+L
B. Ctrl+W
C. Ctrl+G
D. Ctrl+R

Ans: C

Explanation: Ctrl+G opens the Immediate Window (also View > Immediate Window). Ctrl+L is Call Stack, Ctrl+W is Edit Watch and Ctrl+R is Project Explorer.

Q28 | Concept | Debugging Tools

In the Immediate Window, the question mark `?` is a shortcut for:

A. A comment marker
B. The Help command
C. The Print statement
D. The Input statement

Ans: C

Explanation: Typing `? x` is the same as `Print x` and displays the value of x in the Immediate Window.

Q29 | Code Output | Debugging Tools

What will the Immediate Window display for the following entry?

```
? 2 + 3 * 4
```

A. 9
B. 14
C. 24
D. 20

Ans: B

Explanation: Multiplication has higher precedence than addition, so the result is 2 + 12 = 14.

Q30 | Concept | Debugging Tools

Which statement written inside a procedure sends its output to the Immediate Window?

A. Form1.Print
B. MsgBox
C. Debug.Print
D. Immediate.Write

Ans: C

Explanation: Debug.Print writes values to the Immediate Window while the program runs in the IDE, without disturbing the user interface.

Q31 | Concept | Debugging Tools

What happens to `Debug.Print` statements in a compiled .exe file?

A. They raise a run-time error
B. They display a message box
C. They write the output to a text file
D. They are ignored and produce no output

Ans: D

Explanation: Debug.Print is a design-time aid. It is not executed in a compiled executable.

Q32 | Scenario | Debugging Tools

In break mode, the variable `count` holds a wrong value. What is the quickest way to reset it to 0 without editing the code?

A. Change the value in the Form Layout window
B. Edit the entry in the Project Explorer
C. Type a new value in the Quick Watch dialog and press Enter
D. Type `count = 0` in the Immediate Window and press Enter

Ans: D

Explanation: Assignment statements can be executed directly in the Immediate Window, so values can be changed on the fly.

Q33 | Concept | Debugging Tools

Which of the following is NOT allowed in the Immediate Window?

A. Assigning a value to a variable in break mode
B. Calling a Sub procedure
C. Declaring a variable with a `Dim` statement
D. Displaying a value using `?`

Ans: C

Explanation: Declarations such as Dim are invalid in the Immediate pane. It accepts executable statements only, one line at a time.

Q34 | Scenario | Debugging Tools

A Sub procedure named `ShowTotal` has to be tested without clicking through the whole application. While the project is in break mode, what can be done?

A. Drag the procedure into the Watch Window
B. Type `ShowTotal` in the Immediate Window and press Enter
C. Use Edit Watch to run the procedure
D. Press F9 on the procedure name

Ans: B

Explanation: The Immediate Window can call Sub and Function procedures directly, which is useful for quick testing.

Q35 | Concept | Debugging Tools

Which single-line entry correctly prints 1, 2 and 3 on separate lines from the Immediate Window?

A. `For i = 1 To 3: Debug.Print i`
B. `For i = 1 To 3 Debug.Print i Next i`
C. `For i = 1 To 3: Debug.Print i: Next i`
D. `Debug.Print 1 To 3`

Ans: C

Explanation: Block statements must be typed on one line in the Immediate Window, with colons separating the statements.

Q36 | Code Output | Debugging Tools

In `Debug.Print "A", "B"`, the comma between the two items causes B to be displayed:

A. Immediately after A with no space
B. In place of A
C. In the next print zone on the same line
D. On a new line

Ans: C

Explanation: A comma moves the output to the next print zone (tab position); a semicolon would place the items next to each other.

Section 4 11 questions

Debug and Locals Windows

Q37 | Concept | Debugging Tools

What does the Locals Window display in break mode?

A. Only the breakpoints that have been set
B. All variables of the current procedure with their values
C. The list of all forms in the project
D. The sequence of procedure calls that led to the current line

Ans: B

Explanation: The Locals Window automatically lists every variable declared in the current procedure along with its value and type.

Q38 | Concept | Debugging Tools

The Locals Window shows variable values only when the application is in:

A. Compiled .exe mode
B. Run mode without any break
C. Design mode
D. Break mode

Ans: D

Explanation: Variable values can be inspected only while execution is paused, that is, in break mode.

Q39 | Concept | Debugging Tools

Which three columns are displayed in the Locals Window?

A. Variable, Size and Address
B. Name, Scope and Value
C. Expression, Value and Type
D. Expression, Value and Context

Ans: C

Explanation: The Locals Window has Expression, Value and Type columns. The Watch Window has an extra Context column.

Q40 | Scenario | Debugging Tools

While stepping with F8, execution moves into a called procedure. What happens to the Locals Window?

A. It shows the variables of all procedures together
B. It keeps showing the variables of the caller
C. It now shows the variables of the called procedure
D. It closes automatically

Ans: C

Explanation: The Locals Window always reflects the procedure that is currently executing.

Q41 | Concept | Debugging Tools

In the Locals Window of a form, where are the module-level (form-level) variables displayed?

A. They are never displayed
B. Under the `Me` entry, which can be expanded
C. Under the `Project` entry
D. In the Type column

Ans: B

Explanation: The special entry Me represents the current form or class module and holds its module-level variables and controls.

Q42 | Statement-Based | Debugging Tools

Which statement about the Locals Window is correct?

A. It displays only global variables of standard modules
B. A variable's value can be edited in the window during break mode
C. It can be used only in design mode
D. It displays variables of all procedures at the same time

Ans: B

Explanation: In break mode, a value can be changed directly in the Locals Window to test how the program behaves.

Q43 | Concept | Debugging Tools

Which window should be used to follow a variable across several procedures, when the Locals Window shows only the current procedure?

A. Toolbox
B. Project Explorer
C. Properties Window
D. Watch Window

Ans: D

Explanation: A watch expression with a wide context (all procedures, all modules) can be monitored wherever execution is.

Q44 | Concept | Debugging Tools

Which set contains only windows that have buttons on the VB 6.0 Debug toolbar?

A. Toolbox, Properties and Locals
B. Locals, Immediate and Watch
C. Locals, Watch and Toolbox
D. Project, Immediate and Form Layout

Ans: B

Explanation: The Debug toolbar provides buttons for the Locals, Immediate and Watch windows, besides stepping and breakpoint buttons.

Q45 | Concept | Debugging Tools

Which window displays the list of active procedure calls that led to the current statement in break mode?

A. Project Explorer
B. Immediate Window
C. Locals Window
D. Call Stack window

Ans: D

Explanation: The Call Stack dialog (View > Call Stack, Ctrl+L) shows the chain of procedures that are currently active, most recent first.

Q46 | Scenario | Debugging Tools

Procedure A calls B and B calls C. Execution is paused inside C. What does the Call Stack window list?

A. A only
B. All procedures in the project
C. C, B and A
D. A and C only

Ans: C

Explanation: The call stack lists only procedures that are active (called but not yet finished), so it shows C, B and A, with C at the top.

Q47 | Concept | Debugging Tools

Which feature shows the value of a variable in a small tip when the mouse pointer is placed over it in break mode?

A. Auto Data Tips
B. Auto Syntax Check
C. Auto Quick Info
D. Auto List Members

Ans: A

Explanation: Auto Data Tips (Tools > Options > Editor tab) displays the value of the variable under the pointer while debugging.

Section 5 10 questions

Watch Expressions

Q48 | Concept | Debugging Tools

A watch expression is:

A. A line of code highlighted in red by the editor
B. An expression that is evaluated only at compile time
C. A statement that stops the program unconditionally
D. An expression whose value is monitored in the Watch Window during debugging

Ans: D

Explanation: Watch expressions let the programmer track variables or calculated values while the code runs or is stepped through.

Q49 | Concept | Debugging Tools

Which fields are present in the Add Watch dialog box?

A. Name, Value and Type
B. Expression, Context and Watch Type
C. Variable, Scope and Data Type
D. Expression, Breakpoint and Condition

Ans: B

Explanation: Add Watch asks for the Expression, its Context (procedure and module) and the Watch Type.

Q50 | Concept | Debugging Tools

Which are the three Watch Types available in the Add Watch dialog box?

A. Watch Expression, Break When Value Is False, Break When Value Increases
B. Watch Variable, Break When Value Is True, Break On Error
C. Watch Expression, Break At Line, Break When Value Changes
D. Watch Expression, Break When Value Is True, Break When Value Changes

Ans: D

Explanation: The three types are Watch Expression, Break When Value Is True and Break When Value Changes.

Q51 | Scenario | Debugging Tools

Execution should pause automatically as soon as `total > 1000` becomes true. Which watch type should be used?

A. Break When Value Is True
B. Break When Value Changes
C. Watch Expression
D. Quick Watch

Ans: A

Explanation: Break When Value Is True enters break mode when the watch expression evaluates to True (non-zero).

Q52 | Scenario | Debugging Tools

A variable `flag` is being changed unexpectedly by some statement in the program. Which watch type helps to find that statement?

A. Break When Value Is True
B. Set Next Statement
C. Break When Value Changes
D. Watch Expression

Ans: C

Explanation: Break When Value Changes pauses execution whenever the watched value is altered, pointing to the code that changed it.

Q53 | Concept | Debugging Tools

What does `<Out of context>` in the Watch Window indicate?

A. The expression cannot be evaluated at the current execution point
B. The expression has a syntax error
C. The variable has not been declared
D. The watch has been deleted

Ans: A

Explanation: The watch expression was defined for a procedure or module that is not currently executing.

Q54 | Concept | Debugging Tools

Which statement about Quick Watch is correct?

A. It continuously displays all variables of the current procedure
B. It works only in design mode
C. It executes the selected statement in the Immediate Window
D. It shows the current value of the selected expression in a dialog box during break mode

Ans: D

Explanation: Quick Watch (Shift+F9) gives a one-time look at an expression and has an Add button to move it to the Watch Window.

Q55 | Concept | Debugging Tools

In the Add Watch dialog, choosing the context '(All Procedures)' and '(All Modules)' allows the expression to be:

A. Monitored only in the current procedure
B. Evaluated only when the project is compiled to an .exe
C. Evaluated only in the form's Load event
D. Monitored whichever procedure or module is executing

Ans: D

Explanation: The context decides where a watch is evaluated. A wide context avoids the 'Out of context' message.

Q56 | Statement-Based | Debugging Tools

Consider the statements:
1. A watch expression can be a variable or any valid expression such as `x * 2 > 50`.
2. The Watch Window displays Expression, Value, Type and Context.
3. Watch expressions can be added only in design mode.
Which statements are correct?

A. 1 and 2 only
B. 2 and 3 only
C. 1 and 3 only
D. 1, 2 and 3

Ans: A

Explanation: Watches can also be added in break mode (and edited with Edit Watch), so statement 3 is false.

Q57 | Concept | Debugging Tools

Which column appears in the Watch Window but NOT in the Locals Window?

A. Value
B. Context
C. Expression
D. Type

Ans: B

Explanation: The Watch Window shows Expression, Value, Type and Context, whereas the Locals Window has no Context column.

Section 6 10 questions

Testing and Debugging Applications

Q58 | Concept | Debugging Tools

When the VB 6.0 title bar shows [break], the application is:

A. Compiled into an .exe file
B. Being designed
C. Running normally
D. Paused, so code and variables can be inspected

Ans: D

Explanation: The three IDE modes are design, run and break. Break mode suspends execution so the programmer can debug.

Q59 | Concept | Debugging Tools

Which key combination interrupts a running application and switches to break mode?

A. Ctrl+Break
B. Ctrl+F5
C. Shift+F5
D. Ctrl+Alt+Del

Ans: A

Explanation: Ctrl+Break pauses the running program at the current statement. Shift+F5 restarts it.

Q60 | Scenario | Debugging Tools

A program computes an average with `avg = a + b / 2`. For a = 10 and b = 20 it shows 20 instead of 15, with no error message. Which approach best locates the cause?

A. Run the project using Start With Full Compile
B. Step through with F8 and evaluate the expression in the Immediate Window
C. Insert `On Error GoTo 0` before the statement
D. Add `On Error Resume Next` before the statement

Ans: B

Explanation: This is a logical error (operator precedence; it should be (a + b) / 2). Tracing and evaluating expressions reveals it, while error handlers do not.

Q61 | Scenario | Debugging Tools

To record variable values while the program runs at full speed, without stopping at every line, a programmer should insert:

A. Breakpoints
B. `Debug.Print` statements
C. `Resume Next` statements
D. `Set Next Statement` commands

Ans: B

Explanation: Debug.Print writes a trace of values to the Immediate Window while the program keeps running.

Q62 | Scenario | Debugging Tools

In a long procedure, the error is suspected at one particular line. Which is the most efficient way to examine it?

A. Set a breakpoint on that line and run the program with F5
B. Press F8 from the first line of the project
C. Delete the earlier statements and run again
D. Use Set Next Statement on the first line

Ans: A

Explanation: A breakpoint lets everything before the suspected line run normally and pauses just before it.

Q63 | Concept | Debugging Tools

In break mode, pressing F5 will:

A. Continue running until the next breakpoint or the end of the program
B. Execute only the next statement
C. Restart the program from the beginning
D. Remove all breakpoints

Ans: A

Explanation: F5 (Run > Continue) resumes normal execution; F8 would execute only one statement.

Q64 | Scenario | Debugging Tools

You changed the value of a variable in the Immediate Window and want to execute again the statement that has just been run. Which command helps?

A. Show Next Statement
B. Set Next Statement
C. Step Out
D. Clear All Breakpoints

Ans: B

Explanation: Set Next Statement moves the execution arrow back to the earlier line so it can be re-executed.

Q65 | Concept | Debugging Tools

Testing a division program with inputs such as 0 and text is mainly done to:

A. Detect syntax errors in the code window
B. Expose run-time errors for which error handling is needed
C. Turn off the Auto Syntax Check option
D. Reduce the size of the compiled program

Ans: B

Explanation: Unexpected or boundary inputs reveal run-time errors, so that suitable error handlers can then be written.

Q66 | Concept | Debugging Tools

What is the effect of the `Stop` statement placed in code when the program runs in the VB 6.0 IDE?

A. Execution is suspended and the IDE enters break mode
B. All breakpoints are cleared
C. The error handler is disabled
D. The project is closed immediately

Ans: A

Explanation: Stop works like a breakpoint written in the code. It suspends execution so the programmer can examine the program.

Q67 | Concept | Debugging Tools

`Debug.Assert x > 0` enters break mode when:

A. x is declared without a data type
B. The expression x > 0 evaluates to True
C. The expression x > 0 evaluates to False
D. x changes its value

Ans: C

Explanation: Debug.Assert breaks when its condition is False. Like Debug.Print, it is not executed in a compiled .exe.

Section 7 20 questions

Runtime Errors and Error Handlers

Q68 | Concept | Debugging Tools

In a compiled .exe, what happens when a run-time error occurs and no error handler is enabled?

A. VB shows the error message and the application terminates
B. The statement is retried automatically
C. The error is silently ignored
D. VB switches to break mode

Ans: A

Explanation: Without a handler VB displays the standard message and halts. Break mode exists only inside the IDE.

Q69 | Concept | Debugging Tools

Which statement enables an error-handling routine that starts at a line label?

A. On Error GoTo label
B. Err.Raise
C. Resume label
D. Exit Sub

Ans: A

Explanation: `On Error GoTo label` enables the handler; when an error occurs, control jumps to that label.

Q70 | Concept | Debugging Tools

When a run-time error occurs, where does VB look for an enabled error handler?

A. Starting from the first procedure of the project
B. In the current procedure, then up the chain of calling procedures
C. Only in the Form_Load event
D. In the calling procedure only, never in the current one

Ans: B

Explanation: If no handler is found anywhere in the call chain, VB displays the error message and halts execution.

Q71 | Concept | Debugging Tools

Which property of the Err object returns a text message describing the error?

A. Description
B. Number
C. HelpContext
D. Source

Ans: A

Explanation: Err.Number gives the numeric code, while Err.Description gives its message, such as 'Division by zero'.

Q72 | Code Output | Debugging Tools

What message is displayed when the following procedure runs?

```
Private Sub Command1_Click()
    Dim a As Integer, b As Integer
    On Error GoTo ErrHandler
    a = 10: b = 0
    MsgBox a / b
    MsgBox "Done"
    Exit Sub
ErrHandler:
    MsgBox "Error " & Err.Number & ": " & Err.Description
End Sub
```

A. Done
B. Error 13: Type mismatch
C. Error 6: Overflow
D. Error 11: Division by zero

Ans: D

Explanation: a / b raises error 11, control jumps to ErrHandler and the message built from Err.Number and Err.Description is shown.

Q73 | Concept | Debugging Tools

Why is `Exit Sub` normally placed just before the error-handler label?

A. To raise an error deliberately
B. To enable the error handler
C. To stop the handler code from running when no error has occurred
D. To clear the Err object

Ans: C

Explanation: Without Exit Sub, normal execution would flow into the handler code at the end of the procedure.

Q74 | Concept | Debugging Tools

Which statement, used inside an error handler, re-executes the statement that caused the error?

A. Resume Next
B. On Error GoTo 0
C. Resume
D. Exit Sub

Ans: C

Explanation: Resume retries the failing statement, so it should be used only after the cause has been corrected.

Q75 | Concept | Debugging Tools

Which statement resumes execution at the statement immediately after the one that caused the error?

A. Resume
B. Err.Clear
C. Exit Function
D. Resume Next

Ans: D

Explanation: Resume Next skips the faulty statement and continues with the following one.

Q76 | Concept | Debugging Tools

What happens if a new run-time error occurs while an error handler is still active, before Resume or Exit Sub is executed?

A. The same handler runs again automatically
B. VB resumes at the first line of the procedure
C. The same procedure's handler cannot trap it; VB looks for a handler in the calling procedure
D. The error is ignored

Ans: C

Explanation: An active handler cannot handle its own errors. The error is passed up to the calling procedure's handler, or it halts the program.

Q77 | Concept | Debugging Tools

Which method of the Err object generates a run-time error deliberately, for example to test an error handler?

A. Resume
B. Err.Clear
C. Err.Raise
D. Err.Number

Ans: C

Explanation: Err.Raise Number, Source, Description triggers a run-time error as if it had occurred naturally.

Q78 | Concept | Debugging Tools

What does `Err.Clear` do?

A. Raises a new run-time error
B. Clears all entries in the Immediate Window
C. Disables the current error handler
D. Resets the Err object's Number to 0 and Description to an empty string

Ans: D

Explanation: Err.Clear removes the details of the last error so that later checks do not detect it again.

Q79 | Concept | Debugging Tools

In the IDE, which button of the run-time error dialog box takes the programmer to the offending statement in break mode?

A. Debug
B. Cancel
C. Help
D. End

Ans: A

Explanation: Debug shows the statement that caused the error; End terminates the program; Help opens the help topic for that error.

Q80 | Concept | Debugging Tools

Which of the following is the correct form of an error-handler line label?

A. `ErrHandler;`
B. `:ErrHandler`
C. `ErrHandler()`
D. `ErrHandler:`

Ans: D

Explanation: A line label starts at the beginning of the line and ends with a colon.

Q81 | Concept | Debugging Tools

What is the value of `Err.Number` when no run-time error has occurred?

A. 0
B. -1
C. Null
D. 1

Ans: A

Explanation: Err.Number is 0 and Err.Description is an empty string when there is no pending error.

Q82 | Code Output | Debugging Tools

What does this error handler display when `Err.Number` is 11?

```
ErrHandler:
    Select Case Err.Number
        Case 11
            MsgBox "Divide by zero"
        Case 6
            MsgBox "Overflow"
        Case 13
            MsgBox "Type mismatch"
        Case Else
            MsgBox "Unknown error"
    End Select
```

A. Overflow
B. Divide by zero
C. Unknown error
D. Type mismatch

Ans: B

Explanation: Select Case on Err.Number lets one handler respond differently to each error. Number 11 matches the first Case.

Q83 | Concept | Debugging Tools

Which function returns the line number of the statement that caused the last run-time error, if the code lines are numbered?

A. Err.HelpContext
B. Err.Source
C. Erl
D. Err.Number

Ans: C

Explanation: Erl returns the number of the last numbered line executed before the error. It returns 0 if the lines are not numbered.

Q84 | Concept | Debugging Tools

Which function returns the descriptive message for a given run-time error number, for example `Error(11)`?

A. Err.Clear
B. Error
C. Erl
D. MsgBox

Ans: B

Explanation: Error(n) returns the message text for error number n, for example 'Division by zero' for 11.

Q85 | Concept | Debugging Tools

To generate a user-defined error that does not clash with VB's own error numbers, the number passed to `Err.Raise` is usually based on:

A. vbObjectError + n
B. Err.Number - n
C. vbRetry + n
D. vbCritical + n

Ans: A

Explanation: Adding the constant vbObjectError to a small number gives a user-defined error number outside the range reserved for VB errors.

Q86 | Scenario | Debugging Tools

Procedure `Main` has an `On Error GoTo` handler and calls `Calc`, which has no handler. A division by zero occurs inside `Calc`. What happens?

A. Control passes to the error handler of `Main`
B. The error is stored and handled when `Calc` ends
C. VB ignores the error and continues in `Calc`
D. The program always terminates immediately

Ans: A

Explanation: VB searches the calling procedures for an enabled handler. The handler in Main traps the error.

Q87 | Concept | Debugging Tools

Which of the following is a valid way to enable an error handler without using a label name?

A. `On Error GoTo 100:`, where 100 is a variable name
B. `On Error GoTo 100`, where 100 is a line number in the procedure
C. `On Error Resume 100`
D. `On Error Goto Next`

Ans: B

Explanation: On Error GoTo accepts a line label or a line number, and the target must be in the same procedure. GoTo 0 has the special meaning of disabling the handler.

Section 8 6 questions

VB Error-Handling Options

Q88 | Concept | Debugging Tools

Where are the Error Trapping options of the VB 6.0 IDE set?

A. Project > Properties > Make tab
B. Debug > Options
C. Run > Properties
D. Tools > Options > General tab

Ans: D

Explanation: The Error Trapping group is on the General tab of the Options dialog box (Tools > Options).

Q89 | Concept | Debugging Tools

Which are the three Error Trapping options in VB 6.0?

A. Break on All Errors, Break in Class Module, Break on Unhandled Errors
B. Break on All Errors, Break on Syntax Errors, Ignore Errors
C. Break in Form Module, Break in Class Module, Break on Warnings
D. Ignore All Errors, Break on Runtime Errors, Break on Handled Errors

Ans: A

Explanation: These three options control when VB enters break mode because of a run-time error.

Q90 | Concept | Debugging Tools

With the Break on Unhandled Errors option selected, VB enters break mode when:

A. Any run-time error occurs, even when a handler is active
B. A run-time error occurs and no error handler is active
C. A syntax error is typed in the Code window
D. A breakpoint is removed

Ans: B

Explanation: Errors that have an active handler are trapped by it; only unhandled errors interrupt the program in the IDE.

Q91 | Concept | Debugging Tools

Which option makes VB stop at the exact line inside a class module that raised an unhandled error, instead of at the calling line?

A. Break on Unhandled Errors
B. Require Variable Declaration
C. Break in Class Module
D. Break on All Errors

Ans: C

Explanation: Break in Class Module shows the actual statement in the class module where the unhandled error occurred.

Q92 | Concept | Debugging Tools

Besides the Options dialog, Error Trapping can be changed from the Code window's shortcut menu using:

A. The Properties command
B. The Toggle submenu
C. The Watch command
D. The Definition command

Ans: B

Explanation: Right-clicking in the Code window and choosing Toggle offers the three error-trapping settings.

Q93 | Concept | Debugging Tools

Which of the following is NOT a valid form of the `On Error` statement?

A. `On Error Retry`
B. `On Error GoTo 0`
C. `On Error GoTo label`
D. `On Error Resume Next`

Ans: A

Explanation: VB has only three forms: On Error GoTo line, On Error Resume Next and On Error GoTo 0.

Section 9 5 questions

Disabling Error Handlers

Q94 | Concept | Debugging Tools

Which statement disables an enabled error handler in the current procedure?

A. Resume 0
B. Err.Clear
C. On Error GoTo 0
D. On Error Resume Next

Ans: C

Explanation: On Error GoTo 0 turns off error handling, so later errors in that procedure produce the standard run-time error message.

Q95 | Code Output | Debugging Tools

What happens when this procedure runs?

```
Private Sub Test()
    Dim n As Integer, d As Integer, r As Integer
    n = 10: d = 0
    On Error GoTo Oops
    On Error GoTo 0
    r = n / d
    MsgBox "Finished"
    Exit Sub
Oops:
    MsgBox "Handled"
End Sub
```

A. 'Finished' is displayed
B. 'Handled' is displayed
C. Run-time error 13 occurs
D. VB displays the standard run-time error 11 message and the Oops handler does not run

Ans: D

Explanation: On Error GoTo 0 disables the handler before the division, so the error is not trapped.

Q96 | Concept | Debugging Tools

Which Error Trapping option makes VB enter break mode on every run-time error, even when an error handler is enabled?

A. Break When Value Changes
B. Break in Class Module
C. Break on All Errors
D. Break on Unhandled Errors

Ans: C

Explanation: Break on All Errors effectively bypasses error handlers, so the exact line causing an error can be found while debugging.

Q97 | Concept | Debugging Tools

After `On Error GoTo 0` has been executed, how can error handling be enabled again in the same procedure?

A. By using Err.Clear
B. It cannot be enabled again in that procedure
C. By pressing F5
D. By executing another On Error GoTo label or On Error Resume Next statement

Ans: D

Explanation: Any new On Error statement enables error handling again from that point.

Q98 | Concept | Debugging Tools

`On Error GoTo 0` placed in procedure B disables the error handler of:

A. Procedure B only
B. Every procedure in the same module
C. Procedure B and its calling procedure
D. Every procedure in the project

Ans: A

Explanation: An On Error statement affects only the procedure in which it is executed.

Section 10 8 questions

Inline Error Handling

Q99 | Concept | Debugging Tools

Inline error handling is implemented using which statement?

A. Resume label
B. On Error Resume Next
C. On Error GoTo 0
D. On Error GoTo label

Ans: B

Explanation: On Error Resume Next ignores the failure and continues; the code then checks Err.Number itself.

Q100 | Concept | Debugging Tools

In inline error handling, when should the Err object be tested?

A. Only at the end of the procedure
B. Before the statement is executed
C. Only in the Form_Unload event
D. Immediately after the statement that may cause the error

Ans: D

Explanation: Err keeps only the latest error, so it should be checked right after each risky statement.

Q101 | Code Output | Debugging Tools

What does the following code print in the Immediate Window?

```
Dim a As Integer, b As Integer, r As Integer
a = 5: b = 0
On Error Resume Next
r = a / b
r = a + b
Debug.Print Err.Number
```

A. 11
B. A run-time error dialog box appears
C. 0
D. 5

Ans: A

Explanation: Err.Number keeps the value 11 because a later successful statement does not reset the Err object.

Q102 | Concept | Debugging Tools

Why is `Err.Clear` called after an error has been checked in inline error handling?

A. To close the Immediate Window
B. To skip the next statement
C. So that later checks do not detect the old error again
D. To enable the error handler

Ans: C

Explanation: The Err object retains the last error until it is cleared, so a stale value could be mistaken for a new error.

Q103 | Concept | Debugging Tools

What is the main disadvantage of inline error handling?

A. It works only in compiled programs
B. It stops the program at the first error
C. It cannot detect any run-time error
D. Err must be checked after every risky statement, which makes the code longer

Ans: D

Explanation: Forgetting a check makes the error pass silently, and repeated checks clutter the code.

Q104 | Concept | Debugging Tools

How does inline error handling differ from `On Error GoTo label`?

A. Execution continues at the next statement and Err is tested in place, instead of jumping to a separate handler block
B. Inline handling stops the program at the error, whereas GoTo label continues
C. There is no difference
D. Inline handling works only for syntax errors

Ans: A

Explanation: On Error GoTo branches to a labelled routine. Inline handling stays in the normal flow and tests Err.Number after the statement.

Q105 | Code Output | Debugging Tools

What happens when the following code is executed with the text box containing `abc`?

```
On Error Resume Next
n = CInt(Text1.Text)
If Err.Number <> 0 Then
    MsgBox "Invalid number"
    Err.Clear
End If
```

A. n becomes 0 and nothing is displayed
B. Run-time error 13 dialog box appears
C. 'Invalid number' is displayed
D. The program terminates

Ans: C

Explanation: CInt("abc") raises error 13 (Type mismatch). Resume Next skips it and the test of Err.Number then shows the message.

Q106 | Concept | Debugging Tools

Which statement about `On Error Resume Next` and called procedures is correct?

A. It is permanently active until the project is closed
B. It is cancelled only by the Err.Clear method
C. It becomes inactive when another procedure is called, so each procedure needs its own statement
D. It automatically applies to every procedure that is called

Ans: C

Explanation: To use inline handling inside a called procedure, On Error Resume Next must be executed in that procedure also.

Section 11 12 questions

Important and Tricky Questions

Q107 | Match | Debugging Tools

Match List I with List II.
List I: 1. Step Into 2. Step Over 3. Step Out 4. Run To Cursor
List II: a. Ctrl+F8 b. F8 c. Ctrl+Shift+F8 d. Shift+F8

A. 1-a, 2-b, 3-c, 4-d
B. 1-b, 2-d, 3-c, 4-a
C. 1-b, 2-c, 3-d, 4-a
D. 1-d, 2-b, 3-a, 4-c

Ans: B

Explanation: Step Into = F8, Step Over = Shift+F8, Step Out = Ctrl+Shift+F8, Run To Cursor = Ctrl+F8.

Q108 | Match | Debugging Tools

Match List I with List II.
List I: 1. Error 13 2. Error 6 3. Error 11 4. Error 9
List II: a. Overflow b. Subscript out of range c. Type mismatch d. Division by zero

A. 1-c, 2-a, 3-b, 4-d
B. 1-a, 2-c, 3-d, 4-b
C. 1-c, 2-a, 3-d, 4-b
D. 1-d, 2-b, 3-c, 4-a

Ans: C

Explanation: Error 13 = Type mismatch, 6 = Overflow, 11 = Division by zero, 9 = Subscript out of range.

Q109 | Assertion-Reason | Debugging Tools

Assertion (A): Logical errors do not generate error messages in VB.
Reason (R): VB cannot know what result the programmer intended.

A. Both A and R are true and R is the correct explanation of A
B. Both A and R are true but R is not the correct explanation of A
C. A is true but R is false
D. A is false but R is true

Ans: A

Explanation: The compiler checks only grammar and execution rules, not the programmer's intention, so wrong logic goes unreported.

Q110 | Assertion-Reason | Debugging Tools

Assertion (A): `On Error Resume Next` **guarantees that a program is free from errors.**
Reason (R): It only suppresses the error message and moves to the next statement.

A. Both A and R are true and R is the correct explanation of A
B. Both A and R are true but R is not the correct explanation of A
C. A is true but R is false
D. A is false but R is true

Ans: D

Explanation: The error still occurs; Resume Next merely continues execution, so the Err object must be checked.

Q111 | Assertion-Reason | Debugging Tools

Assertion (A): A 'Break When Value Changes' watch can help locate the statement that wrongly modifies a variable.
Reason (R): A watch expression can be monitored only after the application is compiled into an .exe file.

A. Both A and R are true and R is the correct explanation of A
B. Both A and R are true but R is not the correct explanation of A
C. A is true but R is false
D. A is false but R is true

Ans: C

Explanation: Watches work in the IDE during break mode and run mode. They are not related to compiling an .exe.

Q112 | Assertion-Reason | Debugging Tools

Assertion (A): `Exit Sub` **is normally placed just before the error-handler label.**
Reason (R): An error-handler label must end with a colon.

A. Both A and R are true and R is the correct explanation of A
B. Both A and R are true but R is not the correct explanation of A
C. A is true but R is false
D. A is false but R is true

Ans: B

Explanation: Both facts are true, but the colon rule has nothing to do with why Exit Sub is needed (to avoid running the handler by accident).

Q113 | Statement-Based | Debugging Tools

Consider the statements:
1. On Error GoTo 0 disables an enabled error handler.
2. Resume Next re-executes the statement that caused the error.
3. Err.Number is 0 when no error has occurred.
Which statements are correct?

A. 1 and 2 only
B. 1 and 3 only
C. 2 and 3 only
D. 1, 2 and 3

Ans: B

Explanation: Resume re-executes the faulty statement; Resume Next continues with the statement after it, so statement 2 is false.

Q114 | Tricky | Debugging Tools

What happens when this code runs and the error occurs?

```
Private Sub Command1_Click()
    Dim d As Integer
    On Error GoTo Fix
    d = 0
    MsgBox 10 / d
    Exit Sub
Fix:
    Resume
End Sub
```

A. The procedure ends normally after the handler
B. MsgBox shows Infinity
C. MsgBox shows 0
D. The error occurs again and again, causing an endless loop

Ans: D

Explanation: Resume re-executes `10 / d`. Since d is still 0, error 11 occurs repeatedly unless the cause is corrected in the handler.

Q115 | Tricky | Debugging Tools

Which of the following automatically resets the properties of the Err object?

A. A Resume Next statement executed inside an error handler
B. Reading the value of Err.Number
C. Displaying Err.Description in a MsgBox
D. Opening the Quick Watch dialog

Ans: A

Explanation: Err is reset by Resume Next or Exit Sub in a handler, by any On Error statement, or by Err.Clear. Reading its values does not reset it.

Q116 | Tricky | Debugging Tools

A procedure has a correct `On Error GoTo` handler, but in the IDE VB still stops at the faulty statement every time. What is the most likely reason?

A. Error Trapping is set to Break on All Errors
B. Option Explicit is turned off
C. The Err object has been cleared
D. Auto Syntax Check is disabled

Ans: A

Explanation: Break on All Errors enters break mode on every error regardless of handlers. Switch to Break on Unhandled Errors to test the handler.

Q117 | Code Output | Debugging Tools

What is displayed by the following code when it is run?

```
Private Sub Test()
10  Dim a As Integer
20  On Error GoTo H
30  a = 5 / 0
40  Exit Sub
H:
    MsgBox Erl
End Sub
```

A. 11
B. 40
C. 0
D. 30

Ans: D

Explanation: Erl returns the last numbered line executed before the error. The division by zero is on line 30. Line 40 is never reached.

Q118 | Tricky | Debugging Tools

`Err.Raise vbObjectError + 1001` is executed inside a procedure with an active handler. What happens next?

A. A syntax error is reported
B. The Err object is cleared
C. The statement is ignored
D. Control passes to the handler as for a normal run-time error

Ans: D

Explanation: Err.Raise behaves exactly like a naturally occurring error, so the active On Error handler is triggered.

Section 12 10 questions

Mixed OAVS TGT Practice Set

Q119 | Scenario | Debugging Tools

In break mode, the variable `x` holds 5. You type `x = x + 10` in the Immediate Window and press F5. What value of x does the program use when it continues?

A. 5
B. The program restarts and x is reset
C. 15
D. 10

Ans: C

Explanation: Statements typed in the Immediate Window really change the variable, so execution continues with x = 15.

Q120 | Match | Debugging Tools

Match the window with its purpose.
List I: 1. Immediate Window 2. Locals Window 3. Watch Window 4. Quick Watch
List II: a. Shows the current value of one expression in a dialog b. Lists all variables of the current procedure c. Executes statements typed by the programmer d. Monitors chosen expressions with their context

A. 1-c, 2-b, 3-a, 4-d
B. 1-b, 2-c, 3-d, 4-a
C. 1-a, 2-d, 3-c, 4-b
D. 1-c, 2-b, 3-d, 4-a

Ans: D

Explanation: Immediate = executes statements, Locals = all variables of the current procedure, Watch = chosen expressions with context, Quick Watch = one-time value.

Q121 | Code Output | Debugging Tools

What is displayed when the following procedure runs?

```
Private Sub Command1_Click()
    Dim s As String, n As Integer
    On Error GoTo H
    s = "Ten"
    n = s
    MsgBox "OK"
    Exit Sub
H:
    MsgBox Err.Number
End Sub
```

A. Overflow
B. 13
C. 0
D. OK

Ans: B

Explanation: Assigning the text "Ten" to an Integer variable raises error 13 (Type mismatch). The handler displays Err.Number.

Q122 | Concept | Debugging Tools

Which statement transfers control to a specified line label inside the same procedure after an error has been handled?

A. `Err.Resume Cleanup`
B. `Exit Sub Cleanup`
C. `Resume Cleanup` (Cleanup is a line label)
D. `Resume Next Cleanup`

Ans: C

Explanation: `Resume line` continues at the given label or line number, which must be in the same procedure as the error handler.

Q123 | Scenario | Debugging Tools

A user types letters in a text box that should hold a number, and the program stops with run-time error 13. What is the best remedy?

A. Change the watch type to Break When Value Changes
B. Add an error handler, or an inline Err check, around the conversion and show a message
C. Use Step Out when the error appears
D. Set Error Trapping to Break on All Errors

Ans: B

Explanation: Error handling lets the program respond gracefully to invalid input instead of terminating.

Q124 | Scenario | Debugging Tools

While stepping through a procedure with F8, the next line is `Call Calculate`, a long procedure that has already been tested. Which command saves time?

A. Step Over (Shift+F8)
B. Step Into (F8)
C. Set Next Statement
D. Clear All Breakpoints

Ans: A

Explanation: Step Over executes Calculate as one unit and stops at the following line of the current procedure.

Q125 | Error ID | Debugging Tools

Identify the error types in order:
(i) `If x > 5` written without `Then`
(ii) Division by zero when the user enters 0
(iii) `Area = L + B` used instead of `L * B`

A. Runtime, Syntax, Logical
B. Logical, Runtime, Syntax
C. Syntax, Logical, Runtime
D. Syntax, Runtime, Logical

Ans: D

Explanation: (i) breaks the grammar, (ii) fails during execution, (iii) runs but gives a wrong answer.

Q126 | Concept | Debugging Tools

Which property of the Err object identifies the object or application that generated the error?

A. Number
B. Source
C. HelpFile
D. Description

Ans: B

Explanation: Err.Source names the object or application that originally raised the error.

Q127 | Error ID | Debugging Tools

Run-time error 91 'Object variable or With block variable not set' occurs when:

A. A number is divided by zero
B. An object variable is used before an object has been assigned to it
C. A text value is assigned to a numeric variable
D. An Integer variable exceeds 32767

Ans: B

Explanation: Error 91 means the object variable contains Nothing, because no object was assigned to it with Set.

Q128 | Concept | Debugging Tools

Which sequence correctly describes the working of an `On Error GoTo` handler?

A. Error occurs > handler enabled > control jumps to label > Resume or Exit Sub > handler code runs
B. Handler enabled > control jumps to label > error occurs > handler code runs > Resume or Exit Sub
C. Control jumps to label > handler enabled > error occurs > Resume or Exit Sub > handler code runs
D. Handler enabled > error occurs > control jumps to label > handler code runs > Resume or Exit Sub

Ans: D

Explanation: The handler must first be enabled. When an error occurs, control jumps to the label, the handler code runs and then Resume or Exit Sub leaves it.

Quick Answer Key

1-D	**2**-D	**3**-C	**4**-A	**5**-B	**6**-C	**7**-C	**8**-A	**9**-A	**10**-D
11-B	**12**-A	**13**-A	**14**-D	**15**-B	**16**-A	**17**-A	**18**-B	**19**-C	**20**-C
21-A	**22**-D	**23**-D	**24**-C	**25**-B	**26**-B	**27**-C	**28**-C	**29**-B	**30**-C
31-D	**32**-D	**33**-C	**34**-B	**35**-C	**36**-C	**37**-B	**38**-D	**39**-C	**40**-C
41-B	**42**-B	**43**-D	**44**-B	**45**-D	**46**-C	**47**-A	**48**-D	**49**-B	**50**-D
51-A	**52**-C	**53**-A	**54**-D	**55**-D	**56**-A	**57**-B	**58**-D	**59**-A	**60**-B
61-B	**62**-A	**63**-A	**64**-B	**65**-B	**66**-A	**67**-C	**68**-A	**69**-A	**70**-B
71-A	**72**-D	**73**-C	**74**-C	**75**-D	**76**-C	**77**-C	**78**-D	**79**-A	**80**-D
81-A	**82**-B	**83**-C	**84**-B	**85**-A	**86**-A	**87**-B	**88**-D	**89**-A	**90**-B
91-C	**92**-B	**93**-A	**94**-C	**95**-D	**96**-C	**97**-D	**98**-A	**99**-B	**100**-D
101-A	**102**-C	**103**-D	**104**-A	**105**-C	**106**-C	**107**-B	**108**-C	**109**-A	**110**-D
111-C	**112**-B	**113**-B	**114**-D	**115**-A	**116**-A	**117**-D	**118**-D	**119**-C	**120**-D
121-B	**122**-C	**123**-B	**124**-A	**125**-D	**126**-B	**127**-B	**128**-D		